

11.3

MODIFY [SIMD.CTOR]

In [simd.ctor], change:

[simd.ctor]

```
template<class U>
constexpr explicit(see below) basic_vec(U&& value) noexcept;
```

5 Let From denote the type `remove_cvref_t<U>`.

6 *Constraints:* U satisfies ~~`explicitly_convertible_to<value_type>`~~, `convertible_to<value_type>`, and either

- `From` is not an arithmetic type and does not satisfy `constexpr_wrapper_like`,
- `From` is an arithmetic type and the conversion from `From` to `value_type` is value-preserving ([simd.general]), or
- `From` satisfies `constexpr_wrapper_like`, `remove_cvref_t<decltype(From::value)>` is an arithmetic type, and `From::value` is representable by `value_type`.

7 *Effects:* Initializes each element to the value of the argument after conversion to `value_type`.

8 *Remarks:* The expression inside `explicit` evaluates to `false` if and only if U satisfies ~~`convertible_to<value_type>`~~, and either

- ~~`From` is not an arithmetic type and does not satisfy `constexpr_wrapper_like`,~~
 - ~~`From` is an arithmetic type and the conversion from `From` to `value_type` is value-preserving ([simd.general]), or~~
 - ~~`From` satisfies `constexpr_wrapper_like`, `remove_cvref_t<decltype(From::value)>` is an arithmetic type, and `From::value` is representable by `value_type`.~~
-

A

REALLY_CONVERTIBLE_TO DEFINITION

```
template <typename To, typename From>
constexpr bool converting_limits_throws()
{
    try {
        using L = std::numeric_limits<From>;
        [[maybe_unused]] To x = L::max();
        x = L::min();
        x = L::lowest();
    } catch(...) {
        return true;
    }
    return false;
}

template <typename From, typename To>
concept really_convertible_to = std::convertible_to<From, To>
    and not converting_limits_throws<To, From>();
```